

Relaciones entre Clases

- **Asociación:** Es la interacción binaria básica donde una clase utiliza a otra, representada frecuentemente mediante atributos o parámetros.
- **Agregación:** Relación "todo-parte" débil; las partes son independientes y pueden subsistir aunque el objeto principal desaparezca.
- **Composición:** Relación "todo-parte" fuerte con dependencia vital; si el objeto principal se destruye, sus partes también desaparecen.
- **Objetos como atributos:** Técnica que permite usar clases como tipos de datos para campos, favoreciendo la delegación de responsabilidades.
- **Reutilización:** Este enfoque permite que clases como *Persona* y *Empresa* compartan componentes (ej. *Dirección*) sin duplicar código.

Herencia y Jerarquías

- **Relación "Es Un"**: Sistema de categorización jerárquica que organiza los objetos desde lo más abstracto (superclase) a lo más concreto (subclase).
- **Clase Object**: Es la raíz de todas las jerarquías en Java, proporcionando comportamientos comunes a todos los objetos.
- **Mecanismo de Herencia**: La subclase adquiere automáticamente todos los miembros públicos y protegidos de su clase madre.
- **Tipos de jerarquía**: Java soporta herencia única (un solo ancestro directo) y herencia multinivel a través de cadenas de clases.
- **Especialización**: Capacidad de las subclases para añadir nuevos métodos y atributos o reemplazar los existentes para adaptar su comportamiento.

Dinámica de Comportamiento

- **Sobrescritura (Overriding):** Implementación específica en la subclase de un método que ya existe en la superclase, manteniendo la misma firma.
- **Anotación @Override:** Indicador para el compilador que valida la redefinición correcta del método y ayuda a detectar errores.
- **Polimorfismo:** Capacidad de un objeto para comportarse según su tipo real en tiempo de ejecución al ser accedido por una referencia general.
- **Upcasting:** Técnica que permite inicializar objetos de clases hijas utilizando una referencia de la clase madre (ej. `Animal m = new Perro()`).
- **Palabra clave super:** Se utiliza dentro de métodos sobrescritos para invocar la lógica original de la clase madre.

Colecciones Dinámicas

- **Collections Framework:** Conjunto de herramientas más eficiente y flexible que los arrays tradicionales para manejar grupos de datos.
- **List (Listas):** Colecciones ordenadas (como `ArrayList` o `LinkedList`) donde cada elemento posee un índice específico.
- **Set (Conjuntos):** Estructuras desordenadas (como `HashSet`) que prohíben elementos duplicados, ideales para verificar pertenencia.
- **Map (Mapas):** Relacionan una clave única con un valor, funcionando de manera similar a un diccionario o archivo JSON.
- **Iteración:** Uso de la interfaz `Iterable` y el objeto `Iterator` para recorrer colecciones de forma segura, evitando errores de concurrencia.

Organización Modular

- **Paquetes (Packages):** Contenedores que agrupan clases e interfaces relacionadas, organizados físicamente como carpetas en el sistema.
- **Encapsulamiento y Nombres:** Previenen conflictos de nombres permitiendo que clases con el mismo nombre coexistan en paquetes distintos.
- **Nomenclatura Estándar:** Se recomienda el uso del dominio inverso de la empresa para asegurar la unicidad global del paquete.
- **Visibilidad e Importación:** Uso de `public` para acceso externo, `package` para declarar ubicación e `import` para usar clases externas.
- **Separación de Responsabilidades:** Práctica de organizar paquetes según su función (ej. `domain` para lógica, `view` para interfaz).